

A Survey on Transformers in Reinforcement Learning

Presenter: Hongming Zhang

Date: 2023.10.25

Introduction

- Reinforcement Learning
- Transformer
- Transformer in RL
- Discussions

Introduction

- Reinforcement Learning
- Transformer
- Transformer in RL
- Discussions

Reinforcement Learning

A general paradigm for sequential decision-making

- Online learning
- Offline learning
- Meta learning, Multi-agent learning, ...

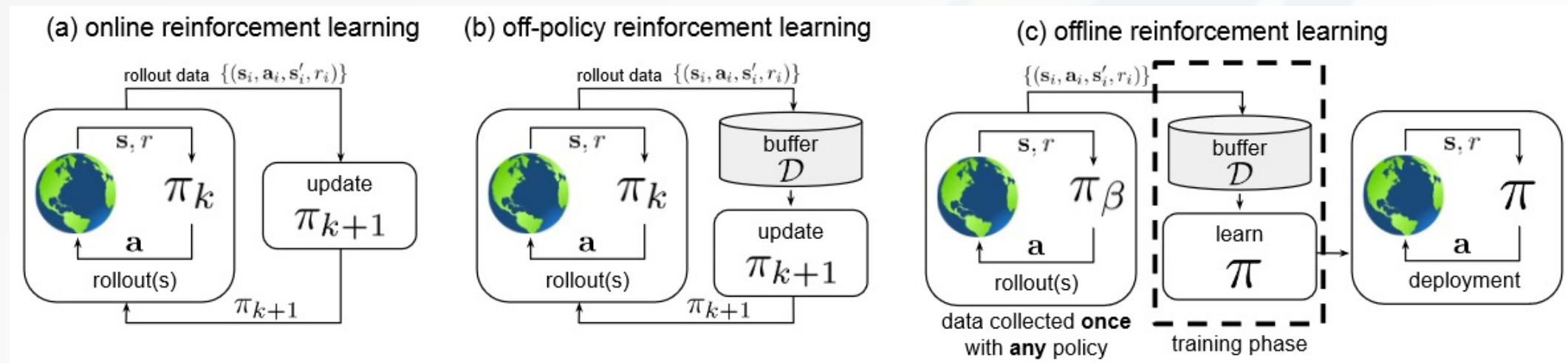
- Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, R, P, \rho_0, \gamma)$$

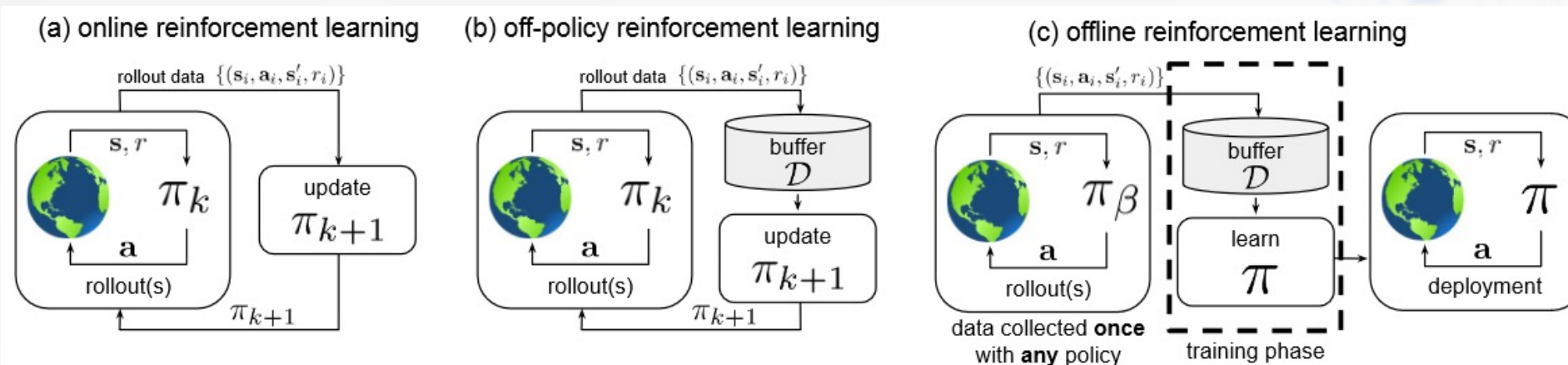
- Objective, maximize the expected cumulative reward:

$$\mathbb{E}[\sum_{t=0}^T \gamma^t r_t]$$

- Value-based, policy-based, model-based, ...



Reinforcement Learning



Main challenge: **sample efficiency** (learning, exploration, ...)

Characteristics: **non-stationary**, **highly sensitive** (network, hyper-parameters, env wrappers, ...)

Introduce **inductive biases**¹: function approximator architectures (CNN, RNN, Transformer, ...), exploration strategies, experience replay, ...

¹<https://zhuanlan.zhihu.com/p/516103074>



- Reinforcement Learning
- Transformer
- Transformer in RL
- Discussions



Transformer

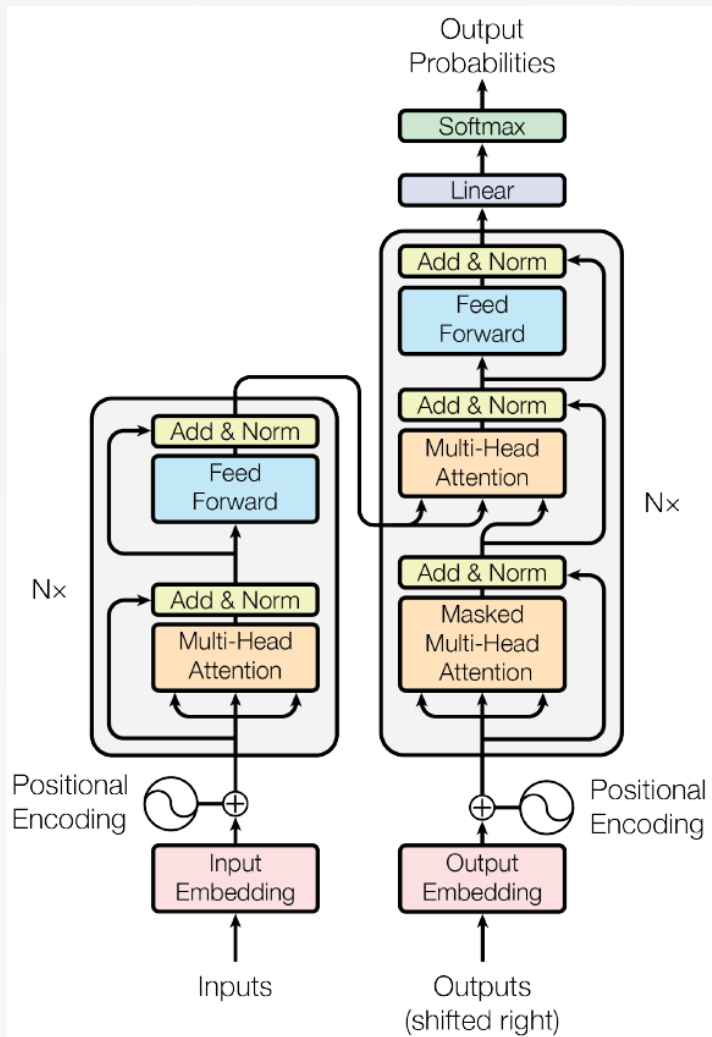
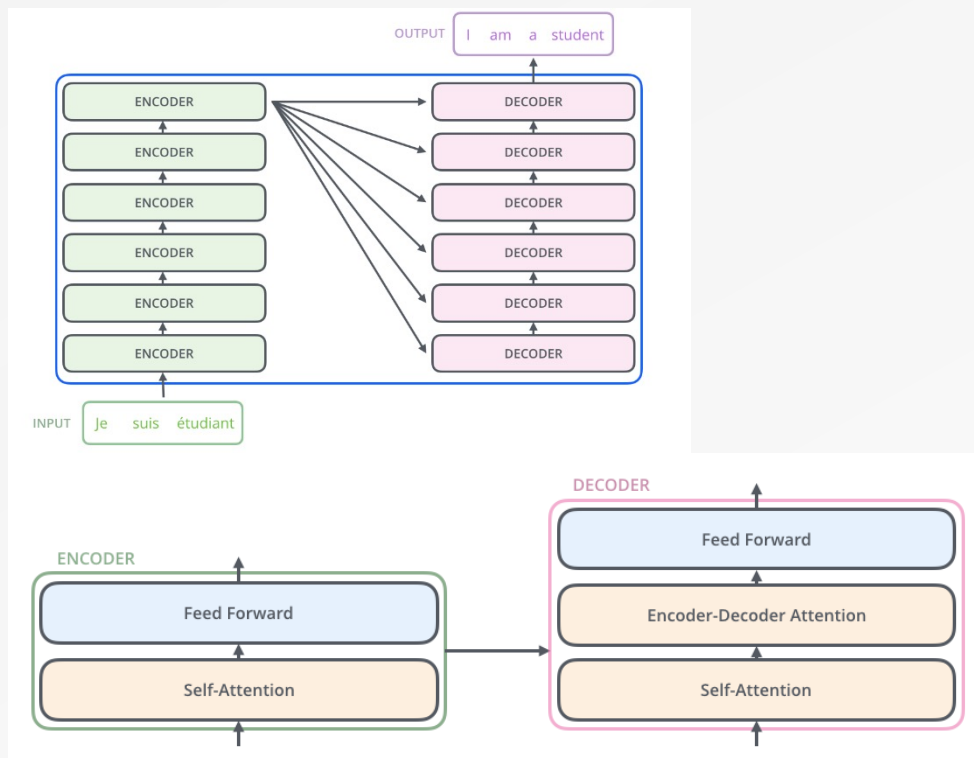


Figure 1: The Transformer - model architecture.

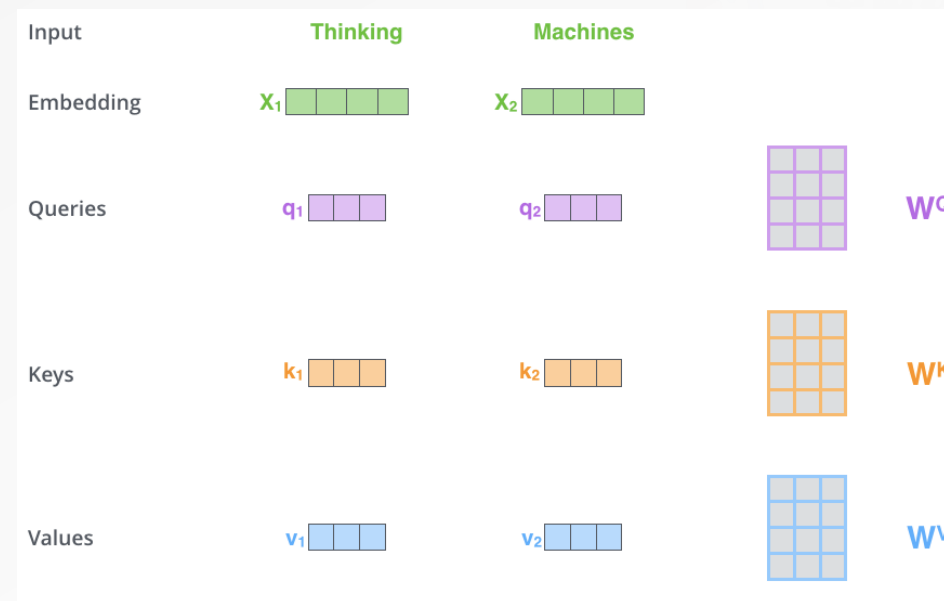
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Characteristics: **sequence modeling**, high computational and memory costs, **less inductive biases** (need much larger amount of training data)

Transformer



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

$$= Z$$

Structure: encoder, decoder

Attention: self-attention, encoder-decoder attention, multi-head attention

Others: tokenization¹, (position/class/time ...) embedding, normalize, residual connection

¹<https://huggingface.co/learn/nlp-course/chapter2/4?fw=pt>

²<http://jalamar.github.io/illustrated-transformer/>

Milestones of Transformer

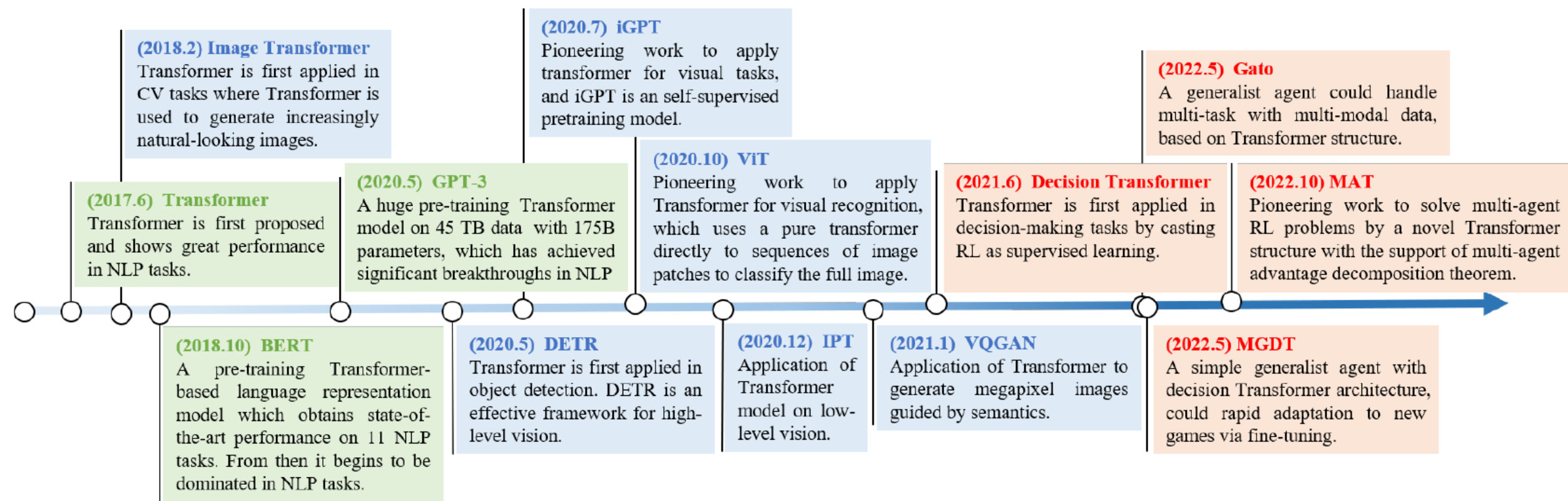


Fig. 1. Key milestones in the development of Transformer. The Transformer-based NLP models are marked in green, the Transformer-based CV models are marked in blue, and the DM Transformer models are marked in red.

Growth of Publications

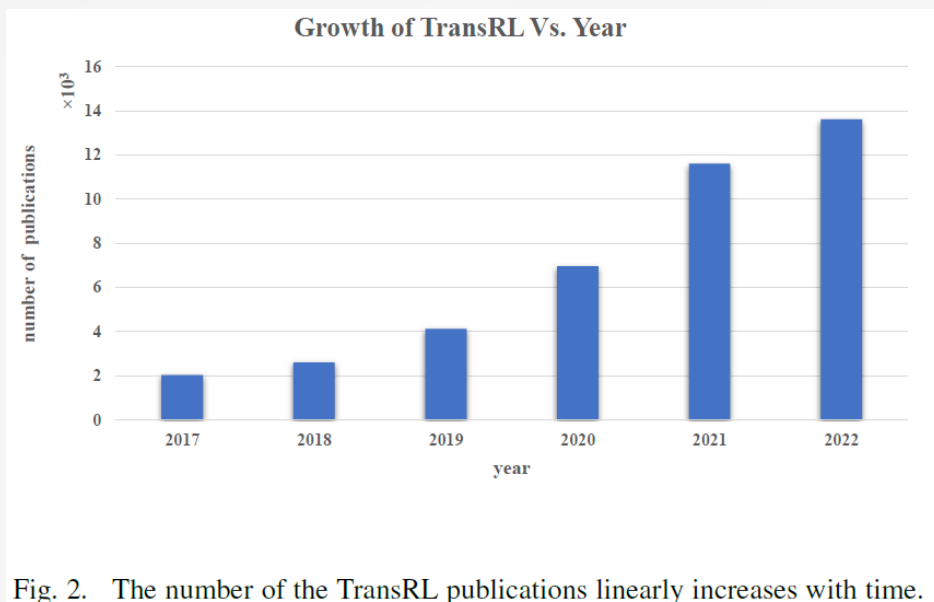


Fig. 2. The number of the TransRL publications linearly increases with time.

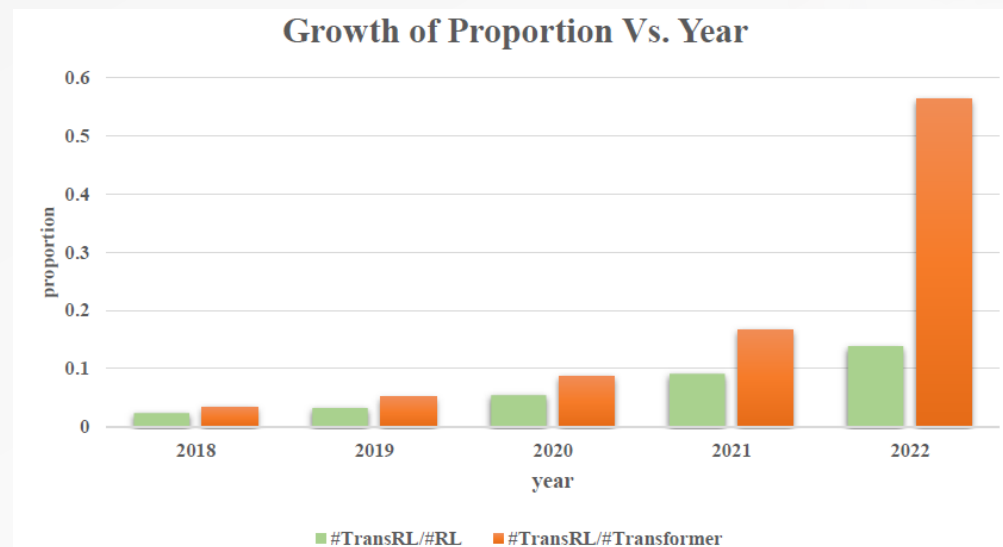
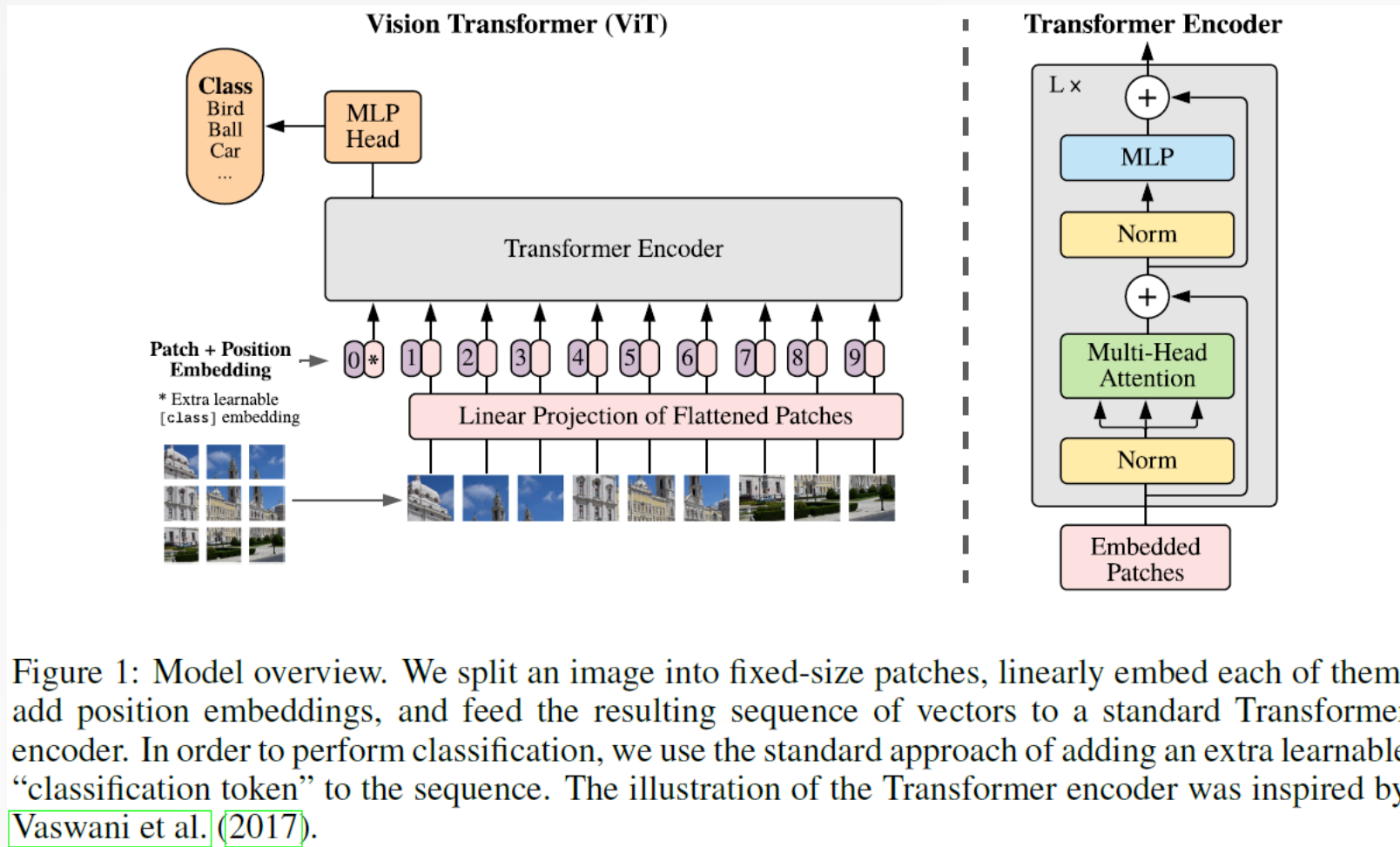


Fig. 3. The proportion of the number of the TransRL publications in the number of the RL publications and the proportion of the number of the TransRL publications in the number of the Transformer publications are both continuously growing with time (where $\#TransRL/\#RL$ represents the proportion of the number of the TransRL publications in the number of the RL researches, and $\#TransRL/\#Transformer$ represents the proportion of the number of the TransRL publications in the number of the Transformer publications).

Vision Transformer (ViT)

A pure transformer applied directly to sequences of image patches can perform very well on [image classification](#) tasks.



Masked Autoencoders (MAE)

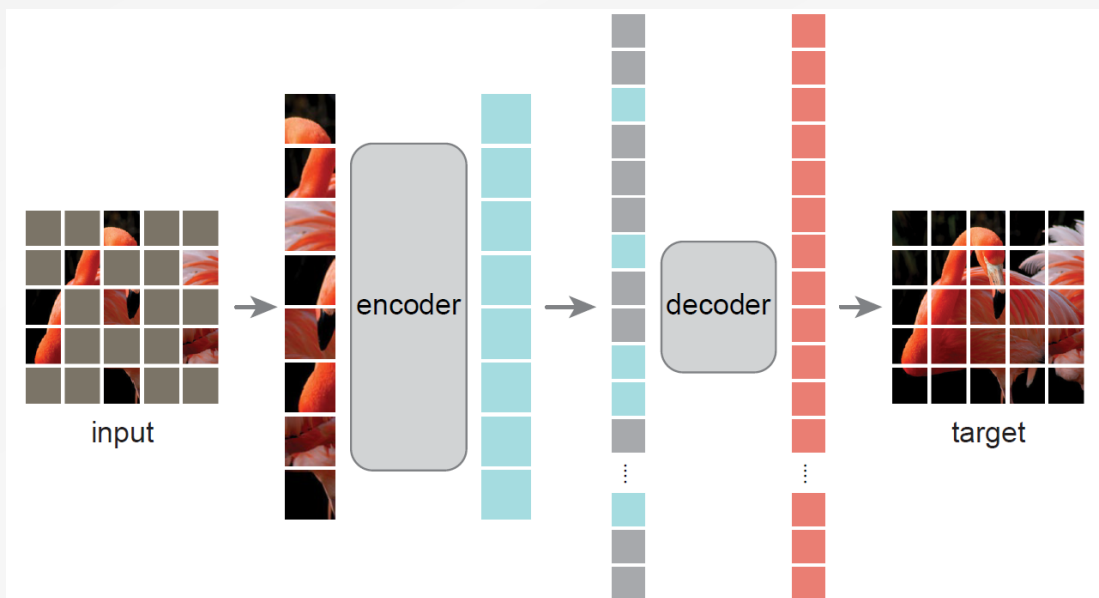


Figure 1. **Our MAE architecture.** During pre-training, a large random subset of image patches (e.g., 75%) is masked out. The encoder is applied to the small subset of *visible patches*. Mask tokens are introduced *after* the encoder, and the full set of encoded patches and mask tokens is processed by a small decoder that reconstructs the original image in pixels. After pre-training, the decoder is discarded and the encoder is applied to uncorrupted images (full sets of patches) for recognition tasks.

Scalable **self-supervised learners** for computer vision.

Mask random patches of the input image and reconstruct the missing pixels.



- Reinforcement Learning
- Transformer
- Transformer in RL
- Discussions



Transformer in RL

- world model

- representation

- policy

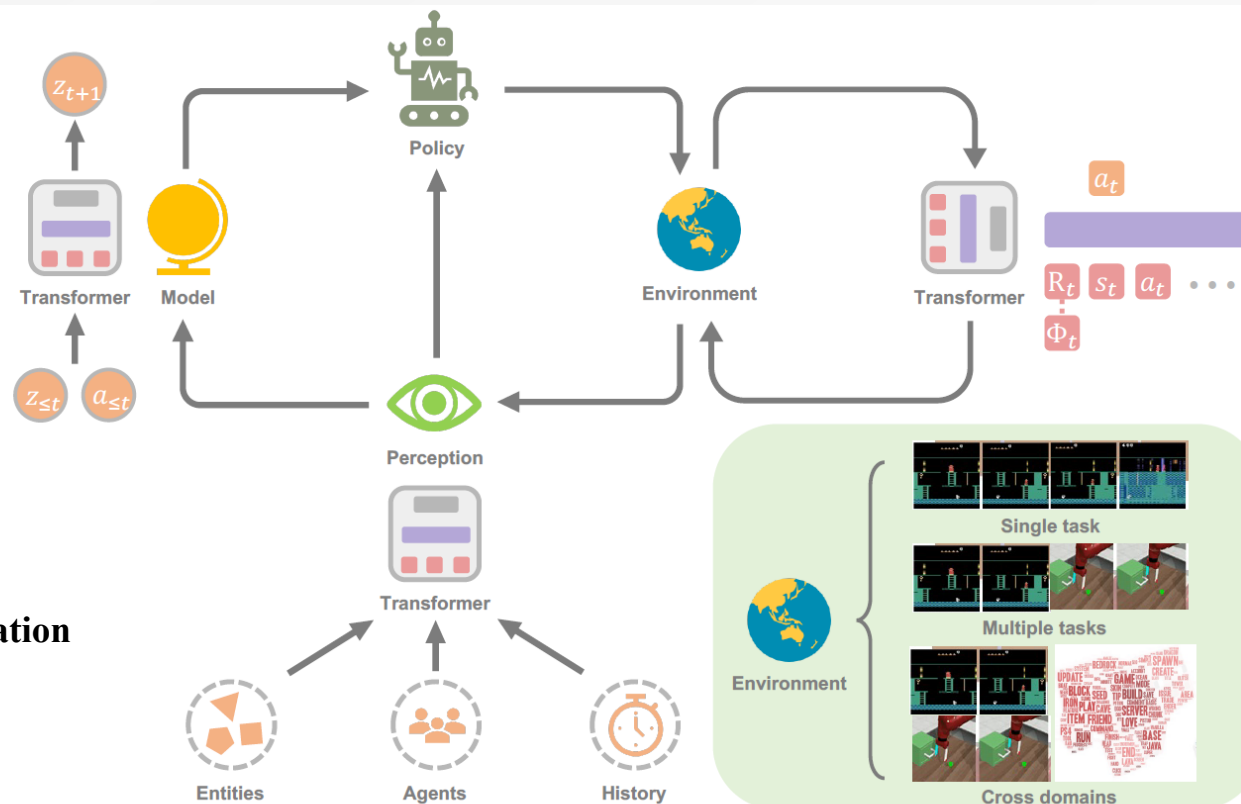


Figure 1: An illustrating example of Transformer-based RL. On the one hand, Transformers can be used as one component in RL. Particularly, Transformers can encode diverse sequences, such as entities, agents, and stacks of historical information; and it is also an expressive predictor for the dynamics model. On the other hand, Transformers can integrate all subroutines in RL and act as a sequential decision-maker. Overall, Transformers can improve RL's learning efficiency in single-task, multi-task, and cross-domain settings.

Milestones of Transformer in RL

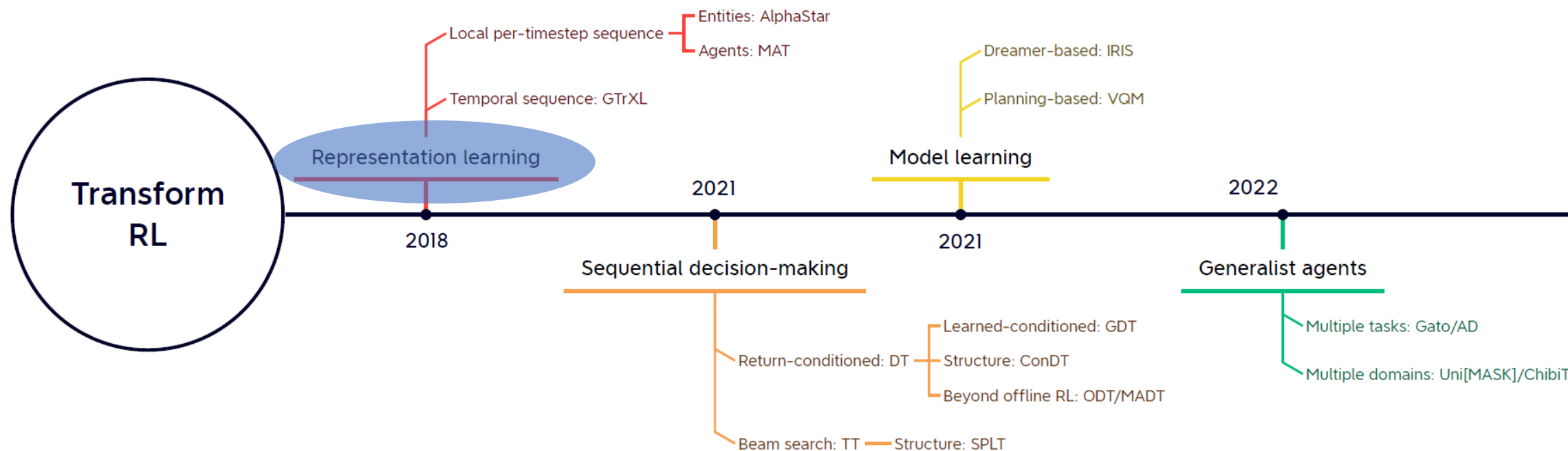
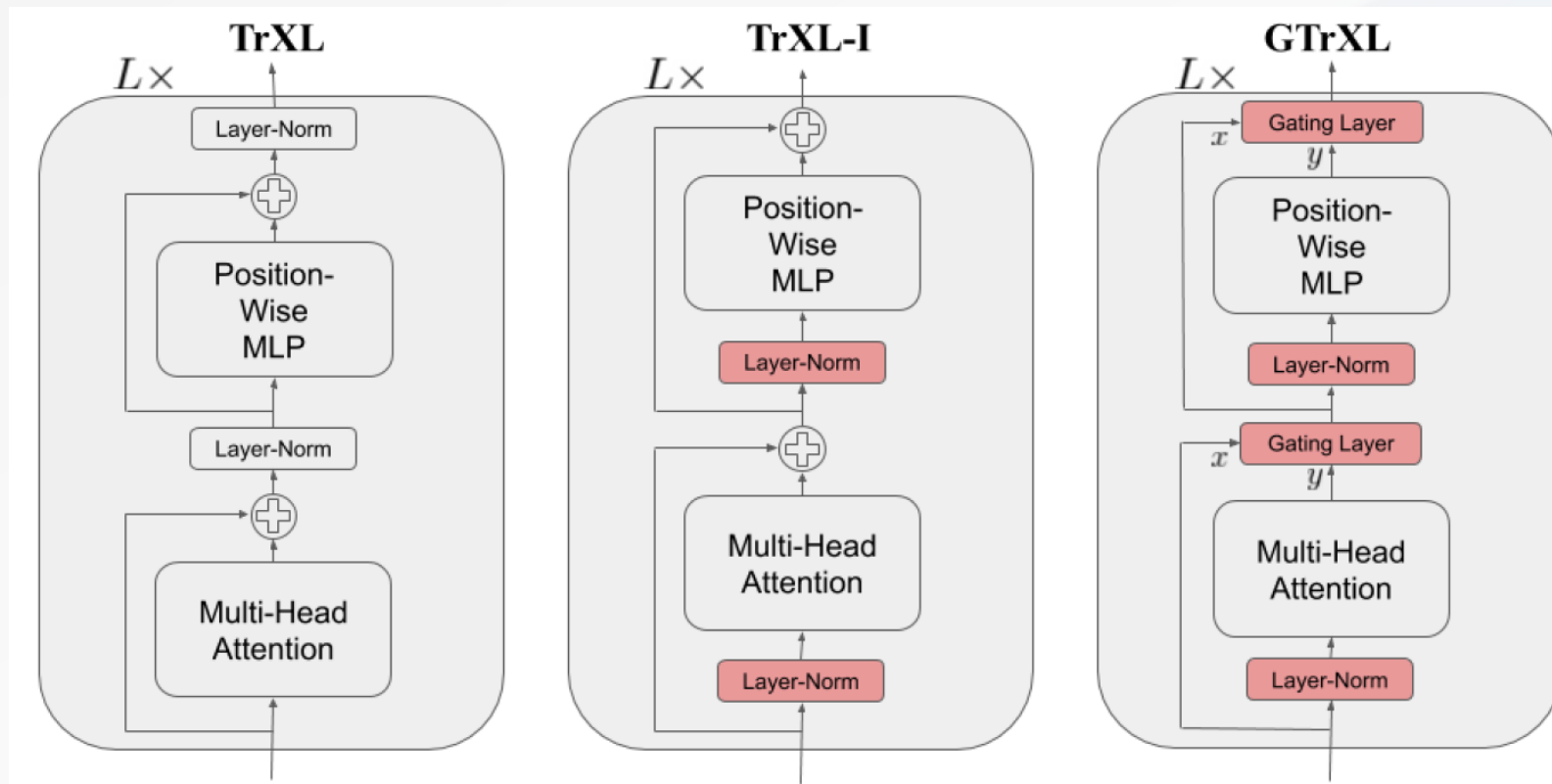


Figure 2: The taxonomy of Transformer-based RL (Transform RL in short). The timeline is based on the first work related to the branch.

Gated Transformer-XL (GTrXL)

The standard transformer architecture is difficult to optimize. Two improvements:

- (1) moves the **layer normalization** to the input stream of the submodules
- (2) additionally adds a **gating layer** in place of the residual connection



Milestones of Transformer in RL

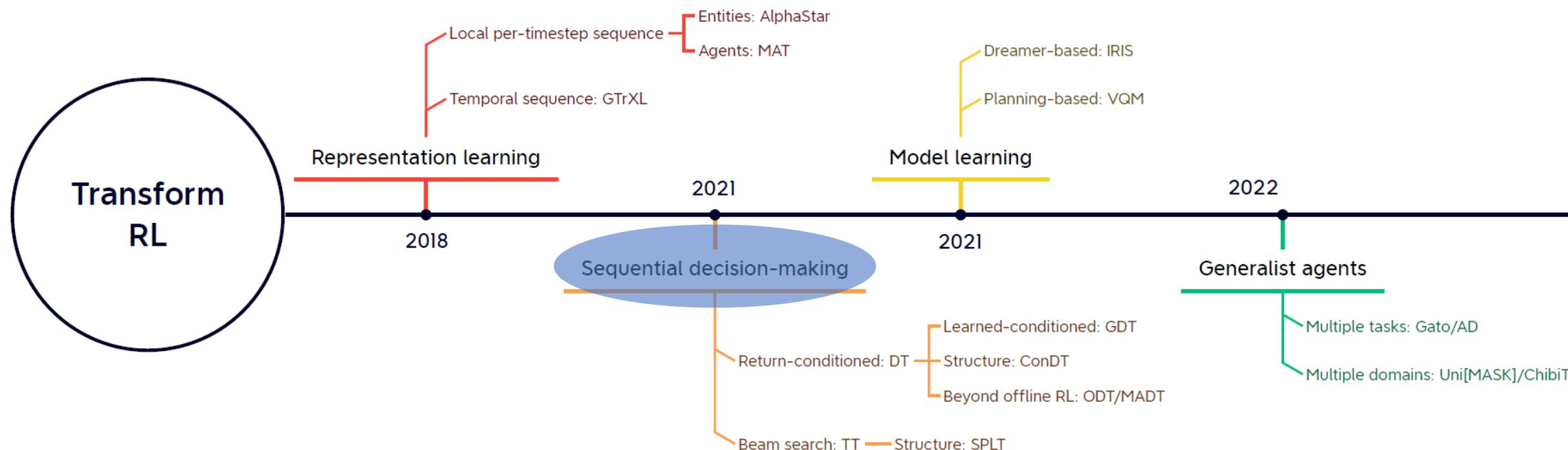
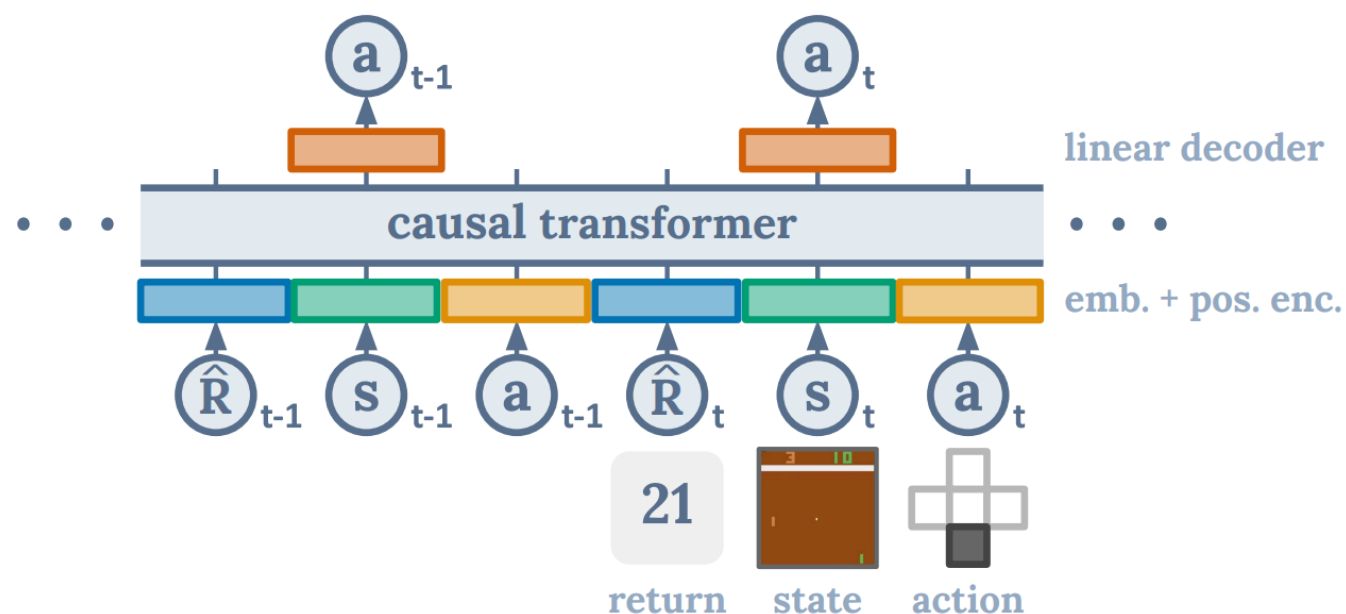


Figure 2: The taxonomy of Transformer-based RL (Transform RL in short). The timeline is based on the first work related to the branch.

Decision Transformer

Casts the problem of RL as **conditional sequence modeling**



- Bypass the need for bootstrapping for long term credit assignment.
- Achieve policy improvement **without the need for dynamic programming**, matches or exceeds the performance of state-of-the-art model-free offline RL algorithms.

Figure 1: Decision Transformer architecture¹. States, actions, and returns are fed into modality-specific linear embeddings and a positional episodic timestep encoding is added. Tokens are fed into a GPT architecture which predicts actions autoregressively using a causal self-attention mask.

Trajectory Transformer

Transformer + beam search

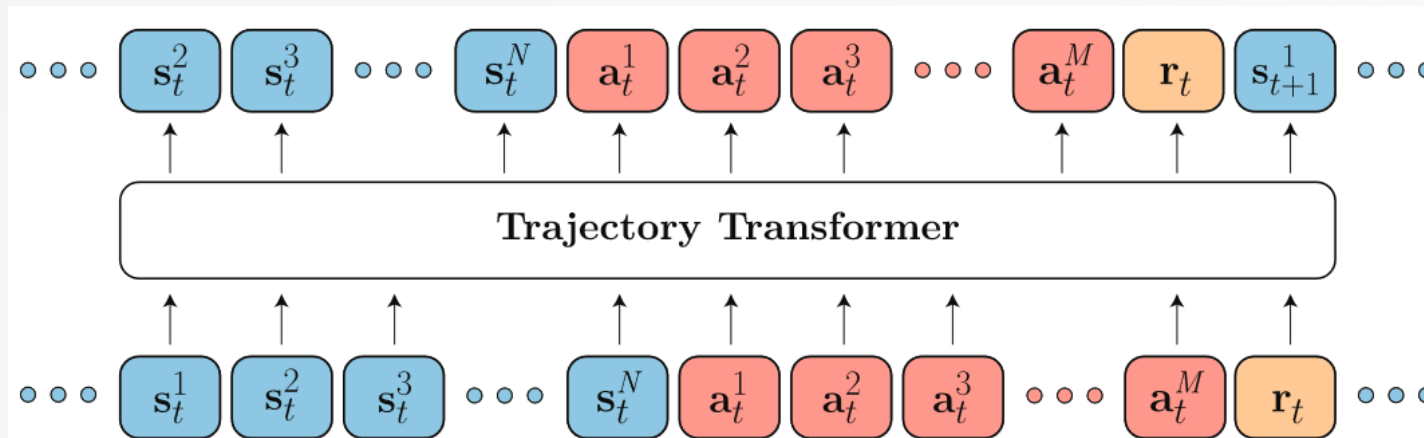


Figure 1 (Architecture) The Trajectory Transformer trains on sequences of (autoregressively discretized) states, actions, and rewards. Planning with the Trajectory Transformer mirrors the sampling procedure used to generate sequences from a language model.

Algorithm 1 Beam search

- 1: **Require** Input sequence $\mathbf{x}$, vocabulary $\mathcal{V}$, sequence length T , beam width B
- 2: **Initialize** $Y_0 = \{ () \}$
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $\mathcal{C}_t \leftarrow \{ \mathbf{y}_{t-1} \circ y \mid \mathbf{y}_{t-1} \in Y_{t-1} \text{ and } y \in \mathcal{V} \}$ // candidate single-token extensions
- 5: $Y_t \leftarrow \underset{Y \subseteq \mathcal{C}_t, |Y|=B}{\operatorname{argmax}} \log P_\theta(Y \mid \mathbf{x})$ // B most likely sequences from candidates
- 6: **end for**
- 7: **Return** $\underset{\mathbf{y} \in Y_T}{\operatorname{argmax}} \log P_\theta(\mathbf{y} \mid \mathbf{x})$

Q-Transformer

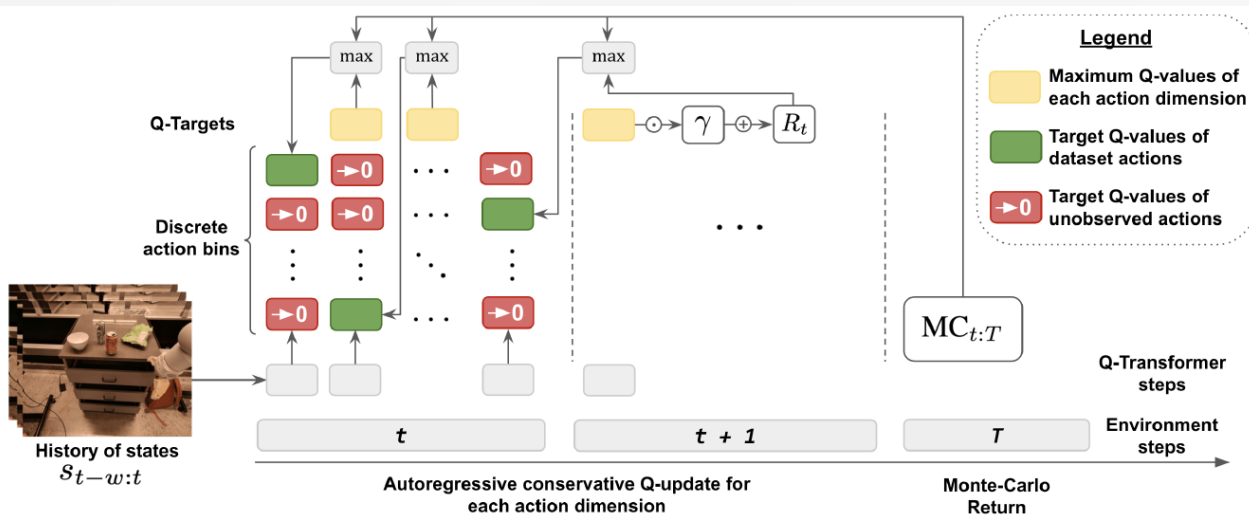


Figure 2: **Q-values update for each action dimension at timestep t .** Given a history of states, we update the Q-values of all bins in all action dimensions. The Q-values of the discrete action bins of the *dataset actions* are trained via the Bellman update (green boxes). The values of action bins not observed in the dataset are minimized towards zero (red boxes). The Q-targets of all action dimensions except the last one are computed using maximization over the next action dimension within the same time step. The Q-target of the last action dimension is computed using the discounted maximization of the first dimension of the next time step plus the reward. We also incorporate Monte Carlo returns by taking the maximum of the computed Q-targets and the return-to-go.

$$Q(s_{t-w:t}, a_t^{1:i-1}, a_t^i) \leftarrow \begin{cases} \max_{a_t^{i+1}} Q(s_{t-w:t}, a_t^{1:i}, a_t^{i+1}) & \text{if } i \in \{1, \dots, d_A - 1\} \\ R(s_t, a_t) + \gamma \max_{a_{t+1}^1} Q(s_{t-w+1:t+1}, a_{t+1}^1) & \text{if } i = d_A \end{cases} \quad (1)$$

Training **multi-task** policies from large **offline** datasets that can leverage both human demonstrations and autonomously collected data.

Discretizing each action dimension and representing the Q-value of each action dimension as separate tokens

Q-learning DT (QDT)

DT+CQL: proposes to use a conservative value function to relabel return-to-go in the dataset, hence combining DT with dynamic programming and improving its stitching capability.

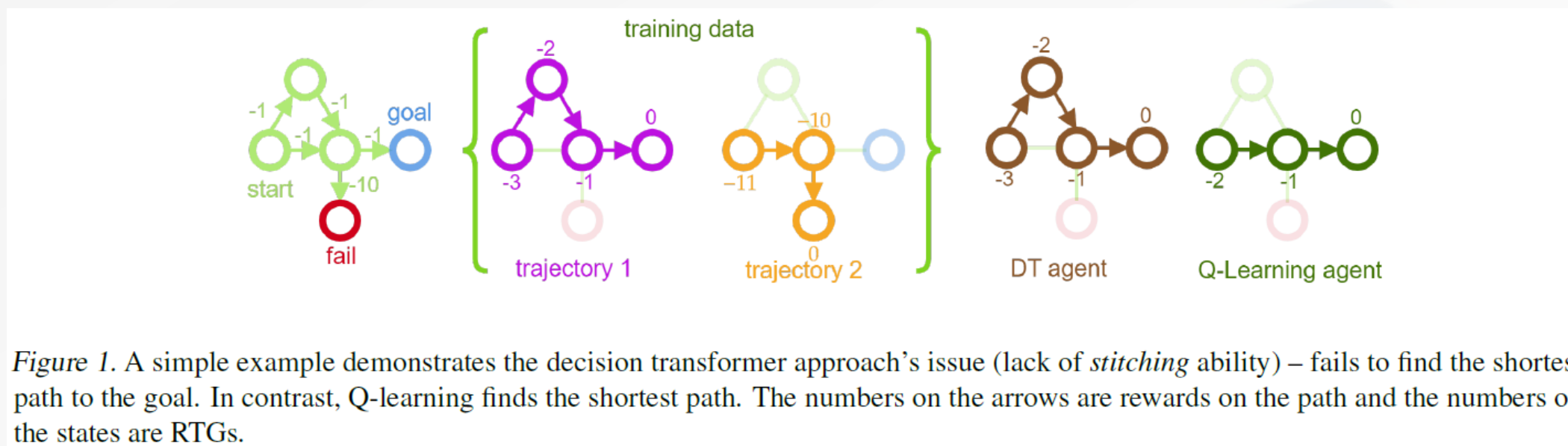


Figure 1. A simple example demonstrates the decision transformer approach's issue (lack of *stitching* ability) – fails to find the shortest path to the goal. In contrast, Q-learning finds the shortest path. The numbers on the arrows are rewards on the path and the numbers on the states are RTGs.

- (1) the value function is learned with Q-learning.
- (2) the offline RL dataset is refined by relabelling the RTG values with the result of Q-learning.
- (3) the DT is trained with the relabelled dataset.

Milestones of Transformer in RL

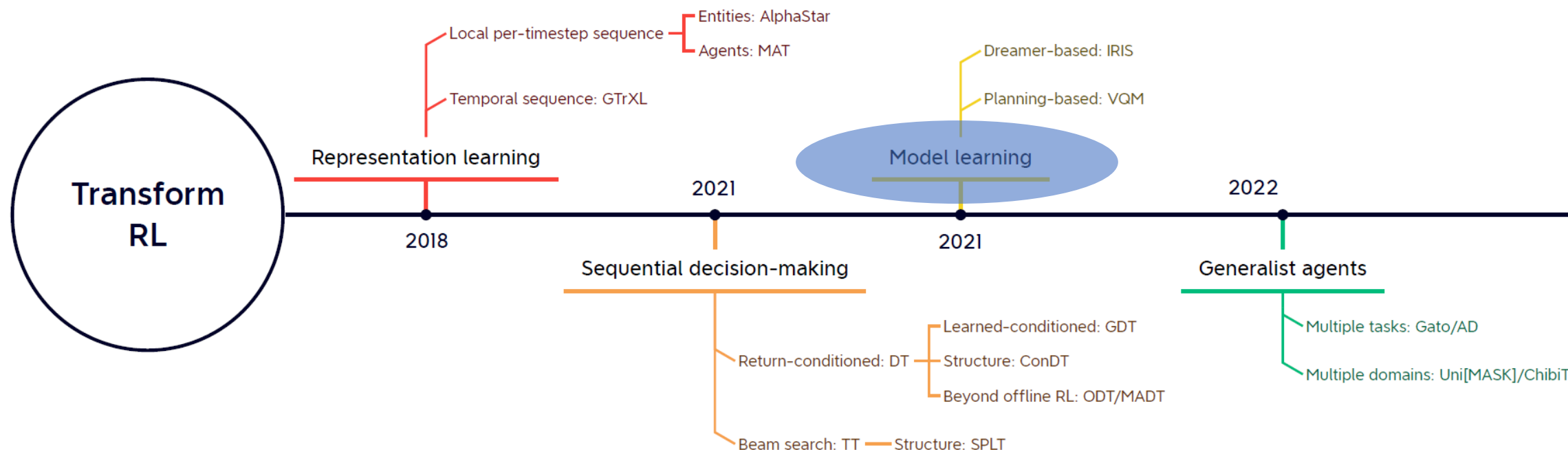


Figure 2: The taxonomy of Transformer-based RL (Transform RL in short). The timeline is based on the first work related to the branch.

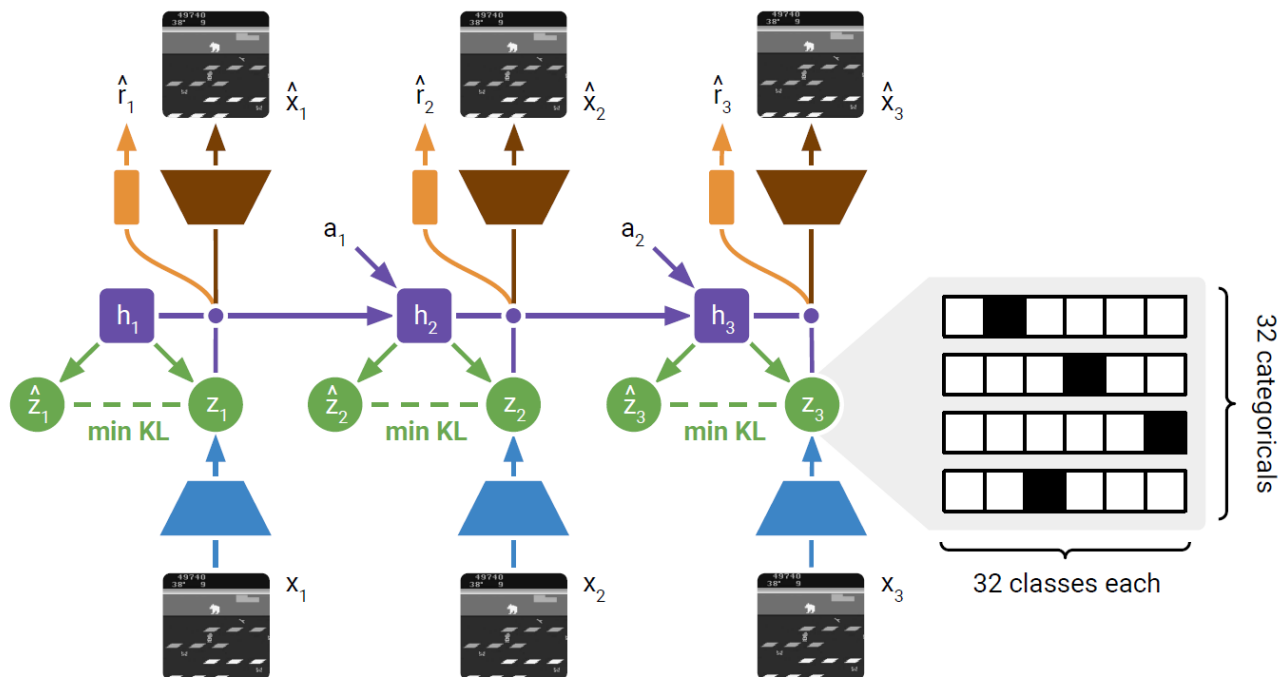


Figure 2: World Model Learning. The training sequence of images x_t is encoded using the CNN. The RSSM uses a sequence of deterministic recurrent states h_t . At each step, it computes a posterior stochastic state z_t that incorporates information about the current image x_t , as well as a prior stochastic state $\hat{z}_t$ that tries to predict the posterior without access to the current image. Unlike in PlaNet and DreamerV1, the stochastic state of DreamerV2 is a vector of multiple categorical variables. The learned prior is used for imagination, as shown in Figure 3. The KL loss both trains the prior and regularizes how much information the posterior incorporates from the image. The regularization increases robustness to novel inputs. It also encourages reusing existing information from past steps to predict rewards and reconstruct images, thus learning long-term dependencies.

Discrete representation of latent states.

RSSM	Recurrent model:	$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$
	Representation model:	$z_t \sim q_\phi(z_t h_t, x_t)$
	Transition predictor:	$\hat{z}_t \sim p_\phi(\hat{z}_t h_t)$
	Image predictor:	$\hat{x}_t \sim p_\phi(\hat{x}_t h_t, z_t)$
	Reward predictor:	$\hat{r}_t \sim p_\phi(\hat{r}_t h_t, z_t)$
	Discount predictor:	$\hat{\gamma}_t \sim p_\phi(\hat{\gamma}_t h_t, z_t)$
Actor:	$\hat{a}_t \sim p_\psi(\hat{a}_t \hat{z}_t)$	
Critic:	$v_\xi(\hat{z}_t) \approx \mathbb{E}_{p_\phi, p_\psi} \left[\sum_{\tau \geq t} \hat{\gamma}^{\tau-t} \hat{r}_\tau \right]$	

Masked World Models (MWM)

MAE+DreamerV2: A visual model-based RL algorithm that decouples visual representation learning and dynamics learning.

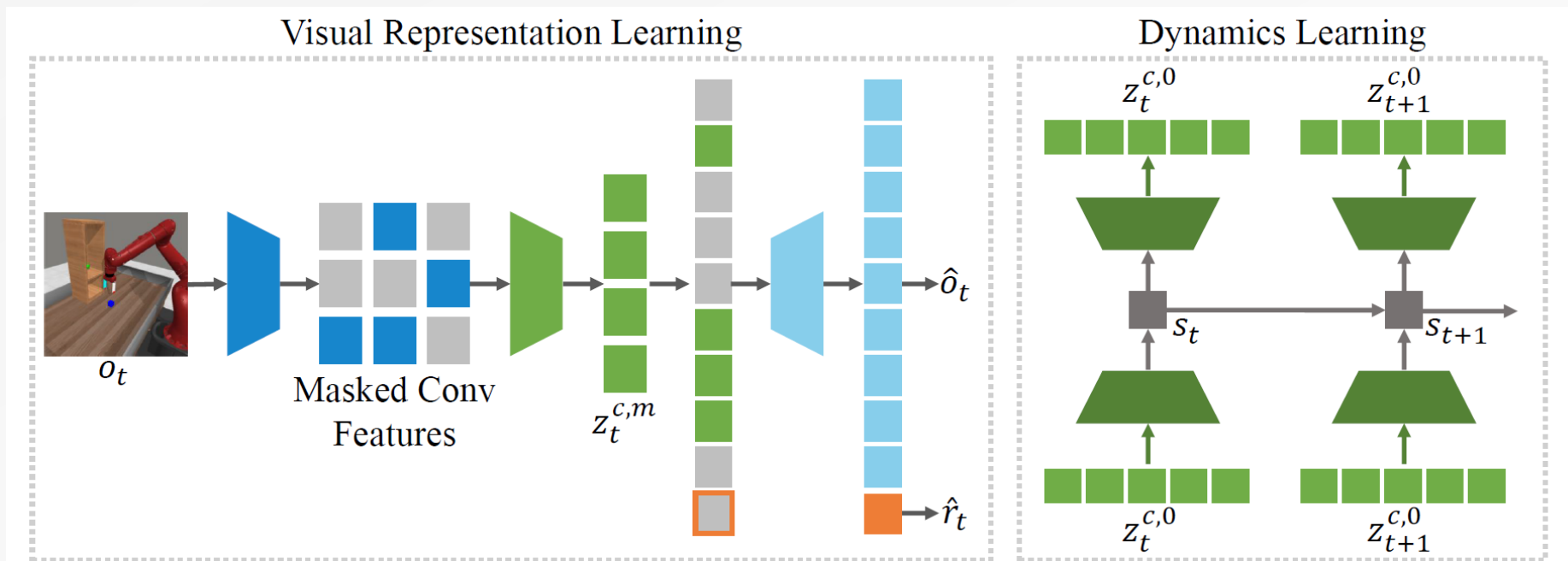
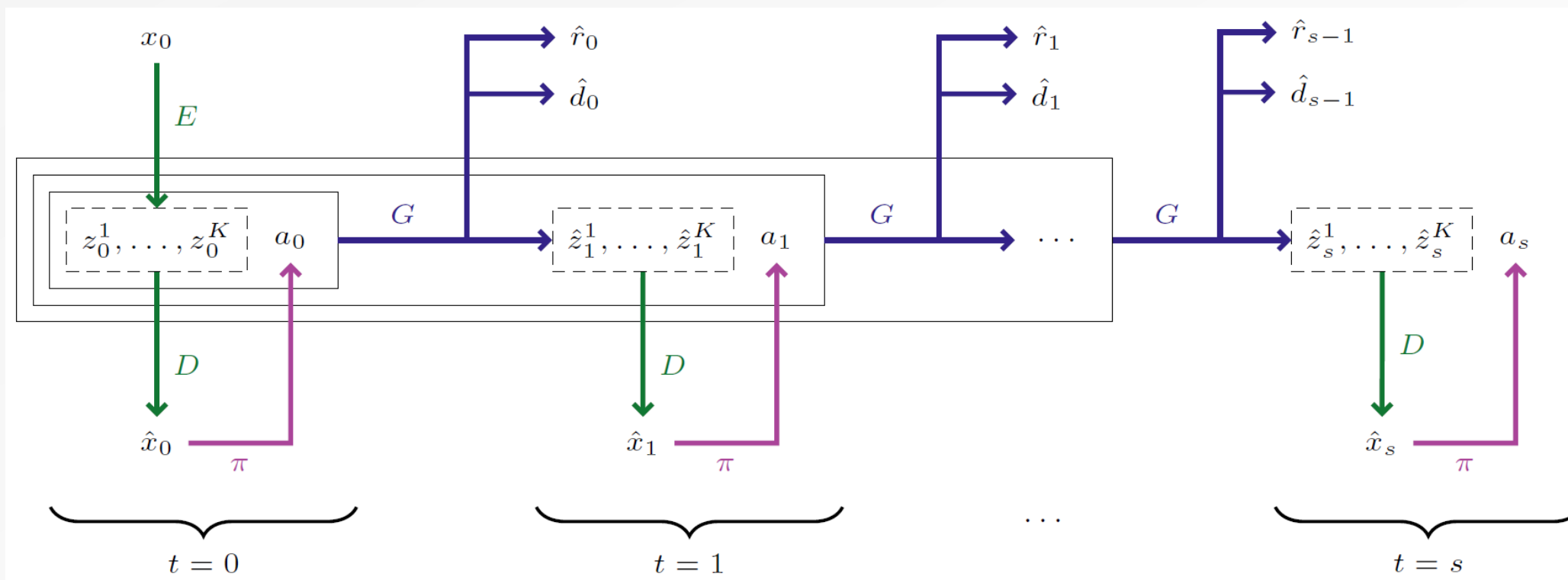


Figure 1: Illustration of our approach. We continually update visual representations and dynamics using online samples collected from environment interaction, by repeating iterative processes of training (Left) an autoencoder with convolutional feature masking and reward prediction and (Right) a latent dynamics model in the latent space of the autoencoder. We note that autoencoder parameters are not updated during dynamics learning.

Imagination with auto-Regression over an Inner Speech (IRIS): **VQ-VAE + Transformer + DreamerV2**

Transformer is a more data-efficient world model compared with Dreamer



Micheli, Vincent, Eloi Alonso, and François Fleuret. "Transformers are Sample-Efficient World Models." ICLR 2023.

Van Den Oord, Aaron, and Oriol Vinyals. "Neural discrete representation learning." Advances in neural information processing systems 30 (2017).

Robine, Jan, Marc Höftmann, Tobias Uelwer, and Stefan Harmeling. "Transformer-based World Models Are Happy With 100k Interactions." ICLR 2023.

Masked Trajectory Models (MTM)

MAE in RL: A generic abstraction for sequential decision making.

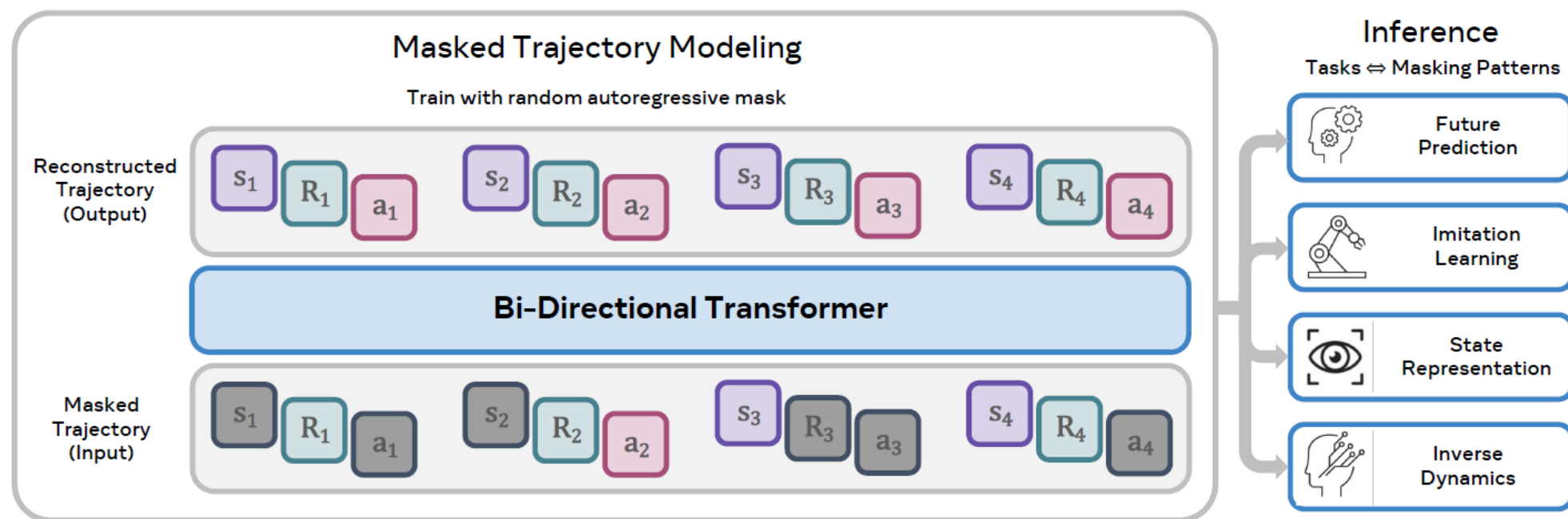


Figure 1. Masked Trajectory Modeling (MTM) Framework. (Left) The training process involves reconstructing trajectory segments from a randomly masked view of the same. (Right) After training, MTM can enable several downstream use-cases by simply changing the masking pattern at inference time. See Section 3 for discussion on training and inference masking patterns.

Milestones of Transformer in RL

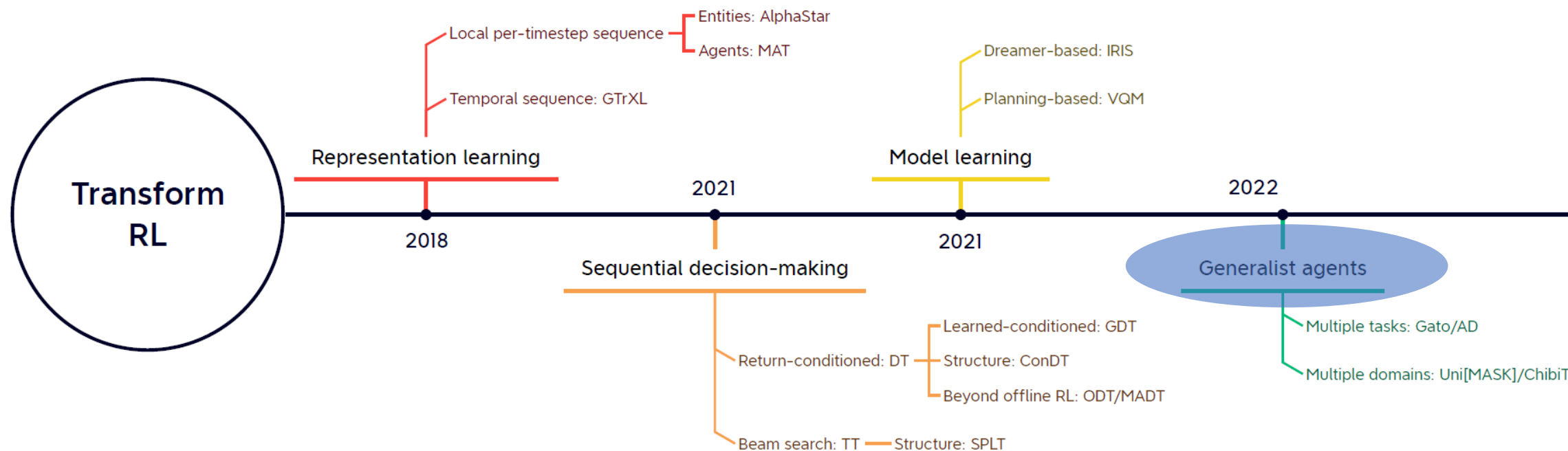


Figure 2: The taxonomy of Transformer-based RL (Transform RL in short). The timeline is based on the first work related to the branch.

Gato

Works as a multi-modal, multi-task, multi-embodiment **generalist** policy.

Serialize all data into a flat sequence of tokens
Text tokens, discrete and continuous values, and actions can be directly set as targets after tokenization.

Gato was trained **offline** in a purely **supervised** manner.

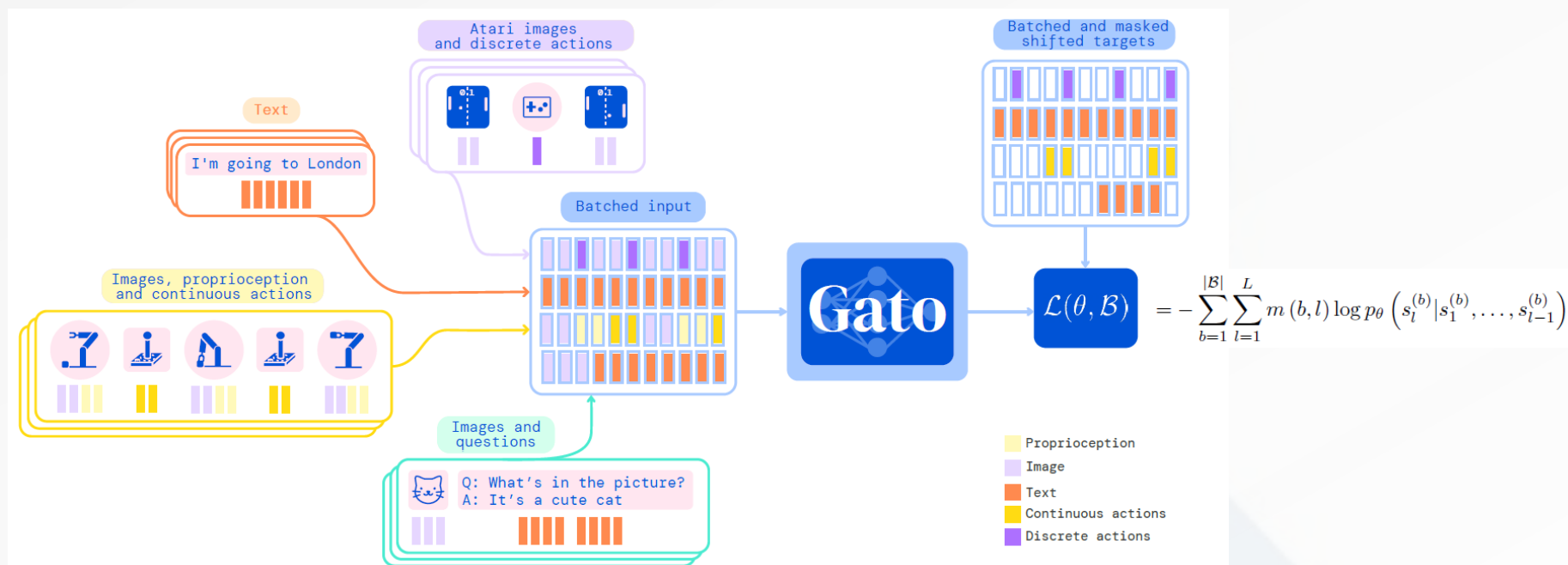


Figure 2: **Training phase of Gato.** Data from different tasks and modalities is serialized into a flat sequence of tokens, batched, and processed by a transformer neural network akin to a large language model. Masking is used such that the loss function is applied only to target outputs, i.e. text and various actions.

Milestones of Transformer in RL

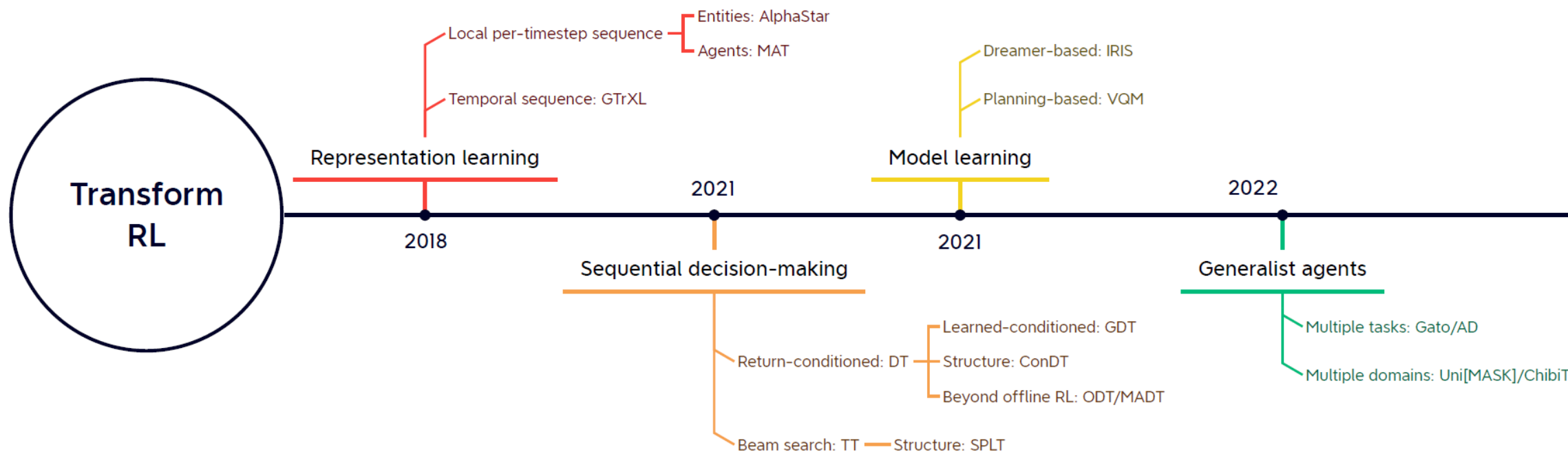
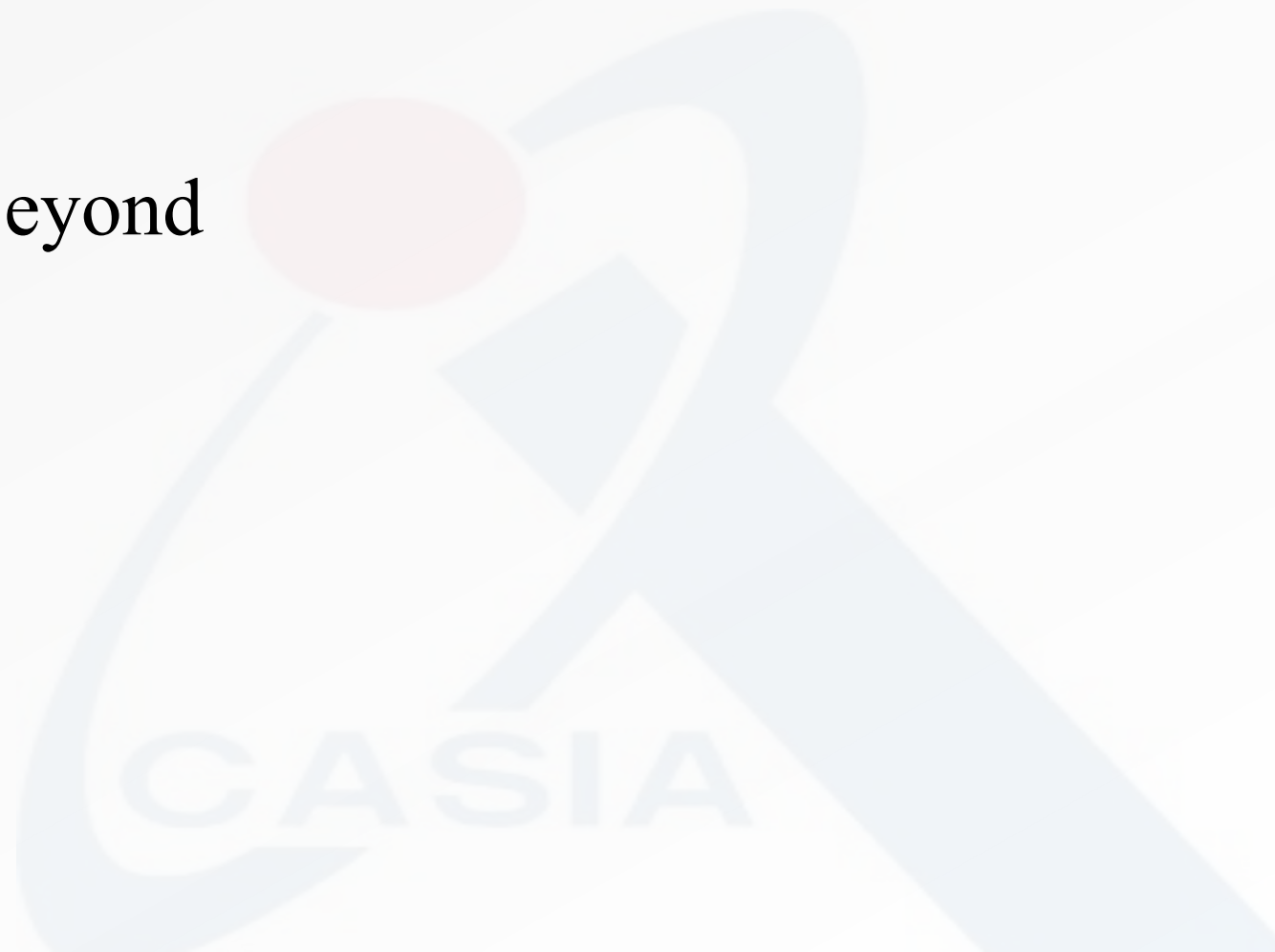


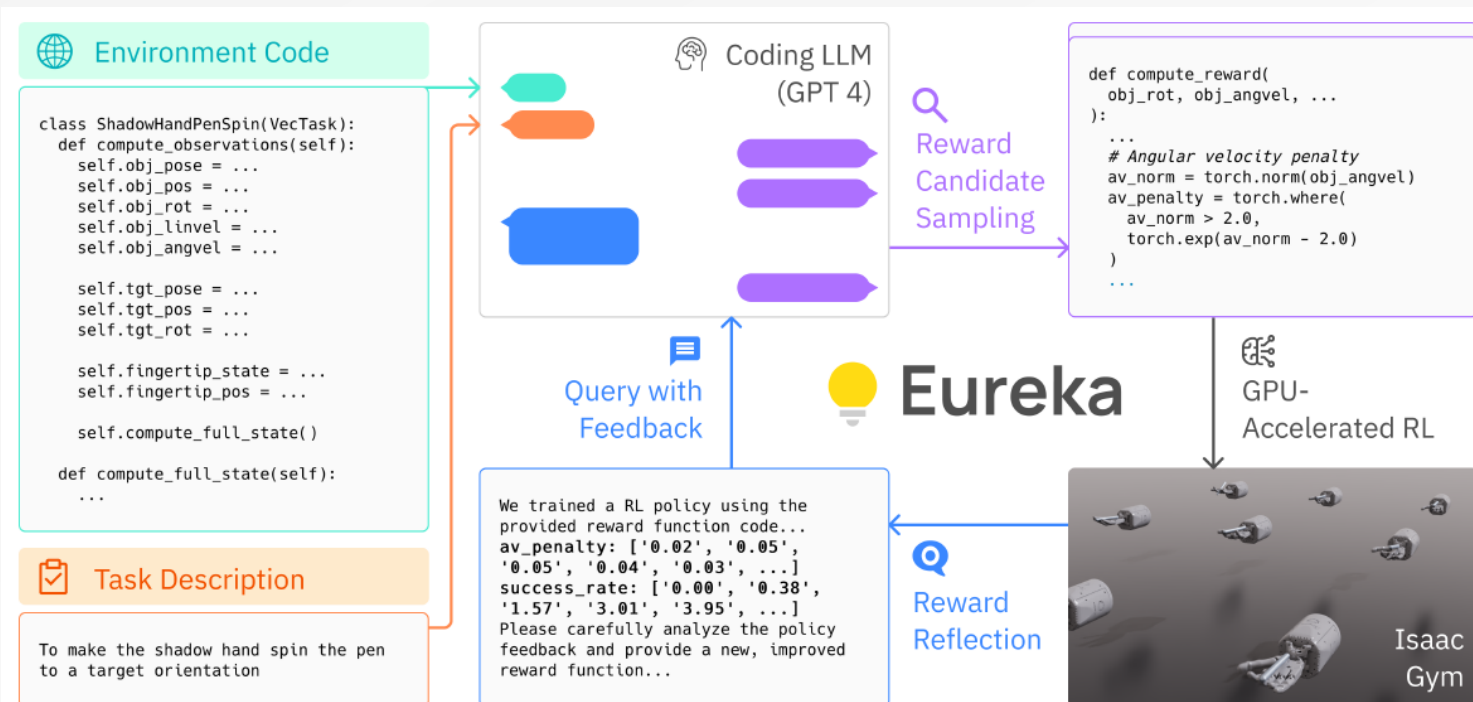
Figure 2: The taxonomy of Transformer-based RL (Transform RL in short). The timeline is based on the first work related to the branch.



Beyond



Human-level **reward design** algorithm powered by LLMs.



Specifying the mutation operator as a text prompt that suggests a few general ways to modify an existing reward code based on a textual summary of policy training

- (1) changing the **hyperparameter** of existing reward components,
- (2) changing the **functional form** of existing reward components, and
- (3) introducing **new reward components**

Figure 2: EUREKA takes unmodified environment source code and language task description as context to zero-shot generate executable reward functions from a coding LLM. Then, it iterates between reward sampling, GPU-accelerated reward evaluation, and reward reflection to progressively improve its reward outputs.

Prompted with related RL paper and chain-of-thought, can produce a competitive performance in RL benchmark tasks

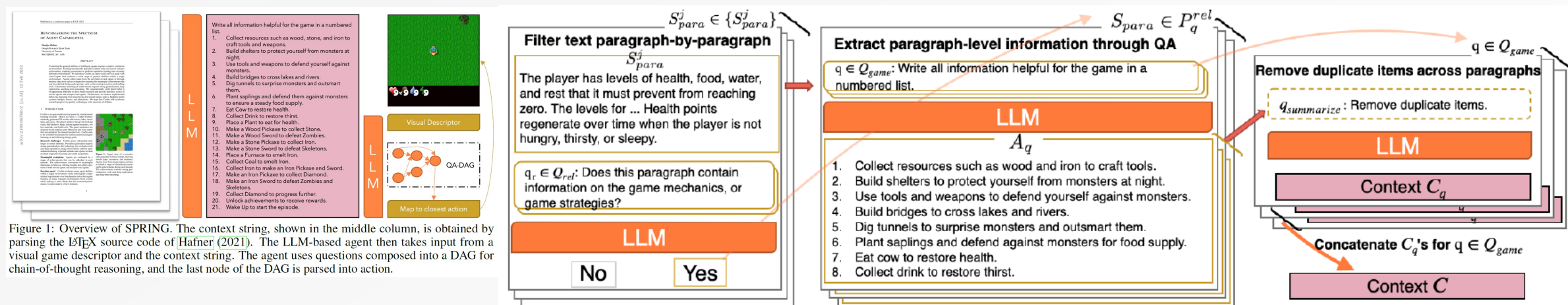


Figure 2: **Paper Studying Moudle**. The 3-step approach for computing C_q from the $\text{\LaTeX}$ source code of Hafner (2021). First, as shown in the left column, for each paragraph we compute LLM answer for all relevancy questions in Q_{rel} , and keep only the relevant paragraphs. Second, as shown in the middle column, we compute paragraph-level LLM answer to q . Third, we summarize the answer into C_q with a summary prompt; we concatenate C_q across $q \in Q_{\text{game}}$ and obtain C .

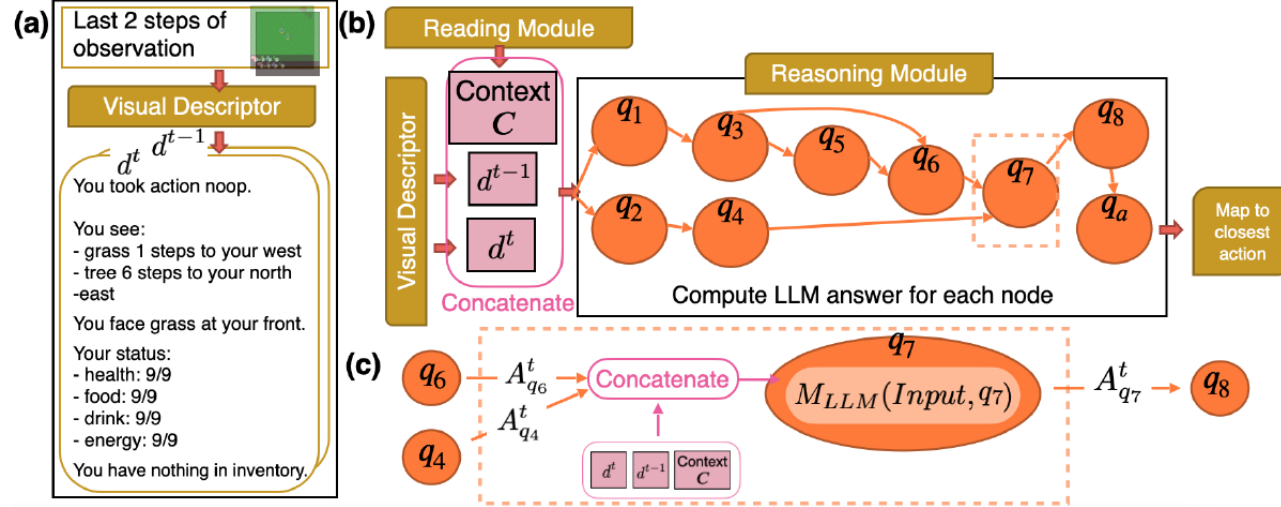


Figure 3: **Reasoning.** (a) The visual descriptor takes the last two gameplay screens as input, and outputs their descriptions in language (d^t, d^{t-1}). (b) SPRING traverses a DAG of questions from Table 1 in topological order. Answer to the final question q_a is mapped to environment action using sub-string matching. (c) The LLM answer for each question (node) is conditioned on the previous 2 steps of observation, the context C , and answers to the immediate parents of the current node.

Node	Question
q_1	List objects in the current observation. For each object, briefly answer what resource it provides and its requirement.
q_2	What was the last action taken by the player?
q_3	For each object in the list, are the requirements met for the interaction?
q_4	Did the last player action succeed? If not, why?
q_5	List top 3 sub-tasks the player should follow. Indicate their priority out of 5.
q_6	What are the requirements for the top sub-task? What should the player do first?
q_7	List top 5 actions the player should take and the requirement for each action. Choose ONLY from the list of all actions. Indicate their priority out of 5.
q_8	For each action in the list, are the requirements met?
q_a	Choose the best executable action from above.

Table 1: List of all 9 questions in Q_{act} . The questions are designed to promote consistent chain-of-thought. Experimentally, we find the LLM robust to different phrasing of the questions.

- Reinforcement Learning
- Transformer
- Transformer in RL
- Discussions

Discussions

- Representation

Differences between Transformer and CNN/RNN

CNN: local, spatial invariance

RNN: sequential, time invariance

Transformer: global, sequential

- Model for POMDP

Transformer for representation + Transformer for dynamics learning

- Online learning using Transformer

Value function/Policy function

- LLM augmented RL

Reward function/Preference-based RL

Generate experience/data augmentation

Guide exploration



THANKS!

